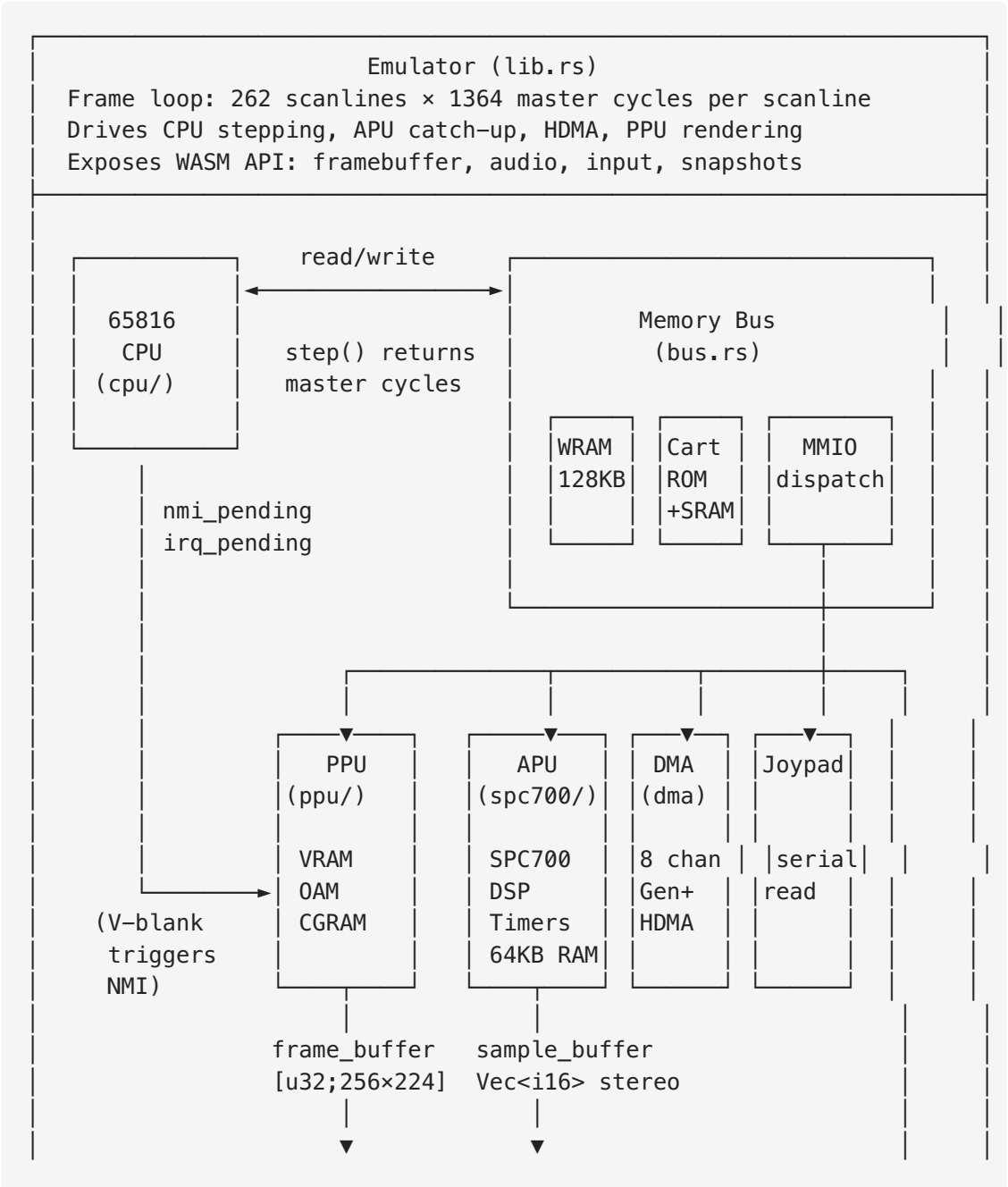
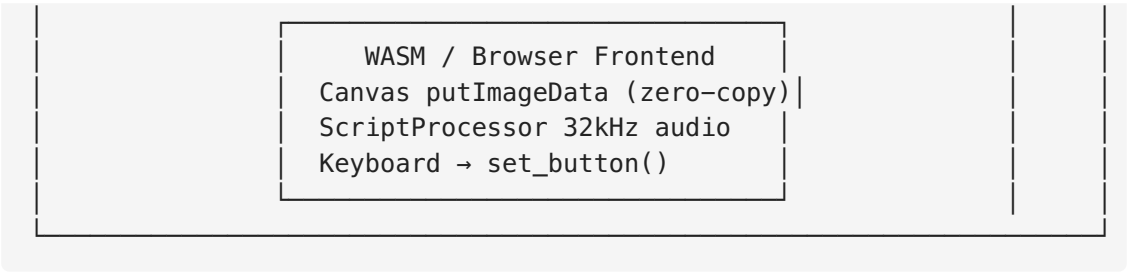
# SNES Emulator — Architecture Documentation

This document describes the architecture of the SNES emulator: seven modules that together recreate the Super Nintendo hardware in software, compiled to both native Rust and WebAssembly.

## System Overview






## Module Inventory

| # | Module | Files | Purpose |
|---|---|---|---|
| 1 | **CPU** | `cpu/mod.rs`, `cpu/addressing.rs`, `cpu/instructions.rs`, `cpu/tables.rs` | 65816 processor emulation |
| 2 | **PPU** | `ppu/mod.rs`, `ppu/render.rs`, `ppu/color.rs` | Picture Processing Unit — video rendering |
| 3 | **APU** | `spc700/mod.rs`, `spc700/cpu.rs`, `spc700/dsp.rs`, `spc700/timers.rs`, `apu.rs`, `spc.rs` | Audio Processing Unit — SPC700 + S-DSP |
| 4 | **Bus** | `bus.rs` | Memory-mapped I/O, address decoding, MMIO dispatch |
| 5 | **DMA** | `dma.rs` + DMA methods in `bus.rs` | General DMA and HDMA (H-blank DMA) |
| 6 | **Cartridge** | `rom.rs` | ROM loading, header parsing, SRAM |
| 7 | **Frontend** | `lib.rs`, `main.rs`, `web/` | Frame loop, WASM bindings, browser rendering + audio |

Supporting files: `joypad.rs` (controller input), `snapshot.rs` (save states).

---

## Module 1: CPU (65816 Processor)

**Files:** `src/cpu/mod.rs`, `src/cpu/addressing.rs`, `src/cpu/instructions.rs`, `src/cpu/tables.rs`

## What It Does

Emulates the Ricoh 5A22, a 65816-based processor running at 3.58/2.68/1.78 MHz depending on memory region. The CPU fetches, decodes, and executes one instruction per `step()` call, returning the number of master cycles consumed.

## Key Structures

**`Cpu`** — All processor state: - Registers: `a` (16-bit accumulator), `x` / `y` (16-bit index), `sp` (stack pointer), `dp` (direct page), `pc` / `pbr` / `dbr` (program counter + bank registers) - `p: StatusRegister` — eight boolean flags (N, V, M, X, D, I, Z, C) - `emulation: bool` — starts `true` (6502 compatibility mode) - `cycles: u64` — cumulative master cycle counter - `nmi_pending` / `irq_pending` — interrupt latches (set externally by Bus/PPU)

**`StatusRegister`** — Processor flags as individual bools. The `m` flag controls 8/16-bit accumulator width; `x` controls index register width. In emulation mode, both are forced to 8-bit.

**`Addr`** — A `{ bank: u8, addr: u16 }` pair returned by addressing mode functions.

## How It Works

```
step(&mut self, bus: &mut Bus) -> u64
│
├── 1. Check STP/WAI halt states → burn 6 cycles
├── 2. Handle pending NMI → push state, load vector, 42 cycles
├── 3. Handle pending IRQ → push state, load vector, 42 cycles
├── 4. [Feature: idle-skip] → detect spin-wait, fast-forward
├── 5. Fetch opcode byte (advances PC)
├── 6. Dispatch: instructions::execute(cpu, bus, opcode) → u8 CPU cycles
└── 7. Return CPU cycles × 6 (master cycle conversion)
```

**Instruction dispatch** is a flat `match opcode { 0x00 => ..., 0x01 => ..., ... }` covering all 256 slots. The `op!` macro sequences address resolution before the operation to satisfy Rust's borrow checker:

```
macro_rules! op {
    ($fn:ident, $addr:expr, $cpu:expr, $bus:expr, $cy:expr) => {{
        let a = $addr;           // resolve addressing (borrows cpu mutably)
        $fn($cpu, $bus, a);      // execute operation (borrows cpu mutably again)
        $cy
    }};
}
```

**Addressing modes** (22 total) live in `addressing.rs`. Each takes `(&mut Cpu, &mut Bus)`, fetches operand bytes (advancing PC), and returns an `Addr` or raw value. Modes include all 65816 variants: direct page, absolute, long (24-bit), stack-relative, indexed indirect, indirect indexed, and indirect long.

**Cycle counting** uses `OPCODE_CYCLES[256]` from `tables.rs` as a base, with branches adding +1 when taken. The fixed ×6 multiplier to master cycles is a simplification — the real SNES has variable-speed bus regions.

## Key Functions

| Function | Location | Purpose |
|---|---|---|
| `Cpu::step()` | `mod.rs:156` | Execute one instruction, return master cycles |
| `Cpu::reset()` | `mod.rs:128` | Load reset vector ($00:FFFC), init registers |
| `instructions::execute()` | `instructions.rs:26` | 256-way opcode dispatch |
| `Cpu::handle_nmi/irq()` | `mod.rs` | Push state, load interrupt vector |
| `Cpu::try_idle_skip()` | `mod.rs` | Feature-gated spin-wait fast-forward |
| `fetch_byte/word/long()` | `mod.rs` | Read from PBR:PC, advance PC |
| `push_byte/word()` | `mod.rs` | Stack operations (page-1 masking in emu mode) |

## Inter-Module Communication

- **CPU → Bus**: Every memory access goes through `bus.read(bank, addr)` / `bus.write(bank, addr, val)`. No direct access to PPU, APU, or cartridge.
- **Bus → CPU**: Interrupt signals are plain booleans (`nmi_pending`, `irq_pending`) set by the frame loop when V-blank occurs.
- **No trait abstraction**: `Bus` is a concrete struct, not a trait object.

## Notable Implementation Details

- All registers are stored as `u16` even in 8-bit mode. The high byte of `a` is the hidden "B accumulator"; 8-bit operations mask with `0xFF00` / `0x00FF` to preserve it.

- `XCE` (exchange carry/emulation) enforces all the mode transition side effects: forcing M/X to 8-bit, zeroing index high bytes, masking SP to page 1.
- `MVN` / `MVP` (block move) transfers one byte per `step()` call, subtracting 3 from PC to re-execute until `a` wraps to `0xFFFF`.
- `SBC` is implemented as `ADC(~operand)` (one's complement trick).
- Decimal mode ( `D` flag) ADC/SBC uses BCD correction with per-nibble carry.

---

## Module 2: PPU (Picture Processing Unit)

**Files:** `src/ppu/mod.rs`, `src/ppu/render.rs`, `src/ppu/color.rs`

### What It Does

Emulates the SNES PPU (two chips: PPU1 + PPU2) which generates the video output. Renders 256×224 pixels per frame using up to 4 tiled background layers and 128 sprites, with color math, window masking, and Mode 7 affine transforms.

### Key Structures

**`Ppu`** — All video state: - **Video memory**: `vram` (64KB), `oam` (544 bytes, 128 sprites), `cgram` (512 bytes, 256 colors) - **Display**: `inidisp` (brightness + forced blank), `bgmode`, `bg: [BgLayer; 4]` - **Mode 7 matrix**: `m7a/b/c/d` (i16 fixed-point), `m7x/y` (rotation center) - **Windows**: `w1/w2_left/right`, per-layer enable + invert + logic operator - **Color math**: `cgwsel`, `cgadsub`, `fixed_color_r/g/b` - **Output**: `frame_buffer: Box<[u32; 256×224]>` in ARGB format - **VRAM access**: prefetch buffer, increment mode, address remapping

**`BgLayer`** — Per-background config: `tilemap_addr`, `tilemap_size` (up to 64×64 tiles), `chr_addr`, `hscroll/vscroll`, `tile_size` (8×8 or 16×16).

**Render-time structs** (not persisted): - `BgPixel` — CGRAM index + priority bit - `ObjPixel` — CGRAM index + priority level (0–3) - `CompositePixel` — resolved 15-bit SNES color + source layer ID + math-exempt flag - `WindowMasks` — `[[bool; 256]; 6]` precomputed per scanline

### How Rendering Works

Rendering is **scanline-based**. The frame loop calls `render_scanline(y)` for visible lines 0–223:

```
render_scanline(y):
│
├── Forced blank? → fill row black, return
├── Compute WindowMasks for all 6 layers (BG1–4, OBJ, Color)
│
├── Mode 7? → render_mode7_scanline() (separate pipeline)
│
├── Modes 0–6:
│   ├── render_bg_scanline() for each enabled BG layer → bg_pixels[4][256]
│   ├── render_obj_scanline() → obj_pixels[256]
│   ├── Composite main screen (layer priority + TM enable + window masks)
│   ├── Composite sub screen (lazy — only if color math needs it)
│   └── Per pixel:
│       ├── Clip-to-black (CGWSEL bits 7–6)
│       ├── Determine if color math applies (region + layer enable)
│       ├── blend_colors() — add/subtract, optional halve, clamp [0,31]
│       └── snes_to_argb(color, brightness) → frame_buffer
```

**Background tile decoding** ( `fetch_bg_pixel` ): 1. Apply scroll offsets to get scrolled (sx, sy) 2. Compute tile coordinates (handling 16×16 tiles and multi-screen tilemaps) 3. Read 2-byte tilemap entry from VRAM: tile number (10-bit), palette (3-bit), priority, h/v flip 4. `decode_tile_pixel` : extract color index from interleaved bitplane data (2/4/8 bpp)

**Sprite rendering** ( `render_obj_scanline` ): - Scans all 128 OAM entries (lower index = higher priority) - Reconstructs 9-bit signed X, selects size from `OBJ_SIZES[8]` table - Always 4bpp, palette mapped to CGRAM 128+

**Mode 7** uses affine matrix multiplication per pixel: the A/B/C/D matrix transforms screen coordinates to VRAM coordinates. Tilemap is 128×128 tiles with interleaved tile/pixel data.

**BG3 high priority** (Mode 1, bit 3 set): BG3 priority-1 tiles render above everything, even OBJ priority 3. This is how A Link to the Past renders the HUD.

## Key Functions

| Function | Location | Purpose |
|---|---|---|
| `Ppu::render_scanline()` | `render.rs:85` | Top-level per-scanline renderer |
| `Ppu::write_register()` | `mod.rs:221` | MMIO write decoder ($2100–$213F) |
| `Ppu::read_register()` | `mod.rs:492` | MMIO read decoder ($2134–$213F) |
| `render_bg_scanline()` | `render.rs` | |

| Function | Location | Purpose |
|---|---|---|
| | | Render one BG layer for one scanline |
| `render_obj_scanline()` | `render.rs` | Render all sprites for one scanline |
| `render_mode7_scanline()` | `render.rs` | Mode 7 affine transform pipeline |
| `composite_layers()` | `render.rs` | Priority-resolve layers into CompositePixel |
| `blend_colors()` | `render.rs:785` | Color math: add/sub/halve/clamp |
| `compute_window_masks()` | `render.rs:712` | Pre-compute 6 × 256 window masks |
| `decode_tile_pixel()` | `render.rs` | Extract pixel from bitplane tile data |

## Inter-Module Communication

- **Bus** → **PPU**: Register writes/reads dispatched from `bus.write()` / `bus.read()` when address is $2100–$213F
- **DMA** → **PPU**: Both general DMA and HDMA write PPU registers directly via `ppu.write_register()`
- **Frame loop** → **PPU**: Sets `ppu.scanline`, calls `render_scanline()`, reads `ppu.frame_buffer`
- **PPU is passive**: It does not generate interrupts, own a clock, or call back into any other module

## What's Implemented vs Missing

| Implemented | Missing / Approximate |
|---|---|
| All 8 BG modes (0–7) | Mosaic effect (register stored, not applied) |
| Mode 7 affine transforms | Mode 7 EXTBG (second BG layer) |
| 2/4/8 bpp tile decoding | Mode 7 fill/repeat behavior (M7SEL) |
| 8×8 and 16×16 tiles | Offset-per-tile (Modes 2/4/6) |
| All 128 sprites, 8 size modes | Interlace / hi-res (Mode 5/6 512-wide) |
| Window masking (W1+W2, logic ops) | Sprite range/time overflow flags |
| Full color math (add/sub/halve) | OPHCT accuracy (hardcoded to 0) |
| Clip-to-black, math regions | Sub-screen sprite compositing in Mode 7 |

| Implemented | Missing / Approximate |
|---|---|
| VRAM address remapping (4 modes) | Direct color mode (CGWSEL bit 0) |
| Master brightness | Mid-scanline HDMA timing |
| BG3 high priority (Mode 1) | Open bus on unimplemented registers |

---

## Module 3: APU (Audio Processing Unit)

**Files:** `src/spc700/mod.rs`, `src/spc700/cpu.rs`, `src/spc700/dsp.rs`, `src/spc700/timers.rs`, `src/apu.rs`, `src/spc.rs`

### What It Does

Emulates the Sony SPC700 audio subsystem — an entirely separate computer inside the SNES with its own CPU, 64KB of RAM, a DSP chip with 8 voices, and 3 hardware timers. Communicates with the main CPU through only 4 bidirectional I/O ports.

### Key Structures

**`Apu`** — Top-level audio unit: - `cpu: Spc700` — the audio processor - `bus: ApuBus` — audio address space (64KB RAM + DSP + timers + I/O ports) - `cycle_debt: i64` — signed debt for cycle-accurate pacing - `dsp_counter: u32` — counts to 32 to trigger sample generation (32 kHz) - `sample_buffer: Vec<i16>` — interleaved stereo output (L,R,L,R,...) - `output_filter: OutputFilter` — analog output stage model (low-pass + high-pass)

**`Spc700`** — Audio CPU: `a`, `x`, `y` (8-bit), `sp` (8-bit, stack at $0100–$01FF), `pc` (16-bit), `psw` (8 flags). Starts at PC=$FFC0 (IPL ROM entry).

**`Dsp`** — S-DSP chip: - `voices: [Voice; 8]` — 8 independent sample channels - `echo_hist_l/r: [i32; 8]` — 8-tap FIR filter history - `noise: i16` — LFSR noise generator - Global counter rate system (31 rates matching blargg)

**`Voice`** — Per-channel state: - `brr_buf: [i32; 12]` — BRR decode ring buffer - `env_level: i32` / `env_phase: EnvPhase` — ADSR/GAIN envelope - `interp_pos: i32` — Gaussian interpolation position - `kon_delay: u8` — 5-sample startup pipeline

**`Timer`** — 3 hardware timers: T0/T1 tick at 8 kHz, T2 at 64 kHz. Each has a divider, 4-bit output counter (read-clears), and configurable target.

## How It Works

**Clock synchronization**: The main frame loop calls `apu.catch_up(master_cycles)` after each CPU step. This converts master cycles to SPC cycles (÷21 with fractional accumulator) and runs `run_cycles()`:

```
run_cycles(target_cycles):
│
│  while cycle_debt < target:
│  │
│  ├── spc700.step(bus) → cycles consumed (2–12)
│  │
│  ├── Every 128 SPC cycles: tick Timer 0, Timer 1
│  ├── Every 16 SPC cycles:  tick Timer 2
│  │
│  └── Every 32 SPC cycles:  dsp.generate_sample(ram)
│       │
│       ├── Process KON (key-on) latches
│       ├── Advance noise LFSR
│       ├── Per voice (×8):
│       │   ├── KON delay pipeline (5-tick startup)
│       │   ├── BRR decode → 4-point Gaussian interpolation
│       │   ├── Envelope × sample → amplitude
│       │   ├── Volume + stereo mix → main L/R accumulators
│       │   ├── Pitch modulation (voice N modulated by voice N-1)
│       │   └── Advance pitch counter, decode next BRR if needed
│       ├── Echo: 8-tap FIR filter, feedback, ring buffer in RAM
│       ├── Final mix: (main × MVOL + echo × EVOL), clamp
│       └── Output filter: low-pass FIR + high-pass DC rejection
```

**BRR (Bit Rate Reduction)**: SNES audio samples are stored as 4-bit ADPCM. Each 9-byte block = 1 header + 8 data bytes = 16 samples. Four IIR filter modes for prediction. The 12-entry ring buffer preserves filter history across block boundaries.

**Envelope**: Two modes — ADSR (Attack/Decay/Sustain/Release with exponential decay) and GAIN (direct level, linear inc/dec, exponential dec, bent-line inc).

## CPU–APU Communication

```
Main CPU side ($2140–$2143):          SPC700 side ($F4–$F7):
  cpu_write(port, val)  ──────────►   bus.read($F4+port) returns val
  cpu_read(port) returns val ◄──────  bus.write($F4+port, val)
```

Each direction has its own latch — writes in one direction don't affect reads in the same direction. The IPL ROM boot handshake uses ports 0/1 to signal readiness ($AA/$BB) and port 0 to begin data transfer ($CC).

### Key Functions

| Function | Location | Purpose |
|---|---|---|
| `Apu::catch_up()` | `spc700/mod.rs` | Master→SPC clock conversion, drives run_cycles |
| `Apu::run_cycles()` | `spc700/mod.rs` | Execute SPC700 + tick timers + generate samples |
| `Spc700::step()` | `spc700/cpu.rs` | Execute one SPC700 instruction |
| `Dsp::generate_sample()` | `spc700/dsp.rs` | One stereo sample at 32 kHz |
| `Dsp::decode_brr_group()` | `spc700/dsp.rs` | Decode 4 BRR samples with IIR filter |
| `Dsp::process_echo()` | `spc700/dsp.rs` | 8-tap FIR echo with feedback |
| `Dsp::update_envelope_step()` | `spc700/dsp.rs` | ADSR/GAIN envelope state machine |
| `Apu::cpu_read/write()` | `spc700/mod.rs` | I/O port access from main CPU |
| `Apu::load_spc()` | `spc700/mod.rs` | Restore state from .SPC file |
| `Timer::tick()` | `spc700/timers.rs` | Advance divider, fire counter |

### Legacy Stub

`apu.rs` contains `ApuStub` — a fake APU that echoes the IPL handshake without running any code. It exists from before the real SPC700 emulator was written and is no longer used in the main path.

---

## Module 4: Memory Bus

**File:** `src/bus.rs`

## What It Does

The Bus is the central interconnect. Every CPU memory access and every DMA transfer flows through `Bus::read()` and `Bus::write()`. It decodes the 24-bit address space (bank:address) and dispatches to the correct hardware.

## Structure

```
pub struct Bus {
    pub cart: Cartridge,              // ROM + SRAM
    pub wram: Box<[u8; 0x20000]>,    // 128KB work RAM
    pub ppu: Ppu,                     // video
    pub apu: Apu,                     // audio
    pub dma: Dma,                     // DMA channels
    pub joypad: Joypad,              // controller

    // CPU-internal registers
    nmitimen: u8, htime: u16, vtime: u16,
    hdmaen: u8, memsel: u8,

    // Hardware multiply/divide
    wrmpya: u8, wrmpyb: u8, wrdiv: u16, wrdivb: u8,
    rddiv: u16, rdmpy: u16,

    // WRAM port (sequential access via $2180)
    wram_addr: u32,                   // 17-bit

    // Status
    vblank: bool, hblank: bool,
    nmi_flag: bool, irq_flag: bool,
    open_bus: u8,

    // Timing
    current_scanline_target: u64,
    pending_dma_cycles: u64,
}
```

## Memory Map (LoROM)

The SNES has a 24-bit address space (16MB). Banks $80–$FF mirror $00–$7F via `bank & 0x7F`.

```
Bank    Address         Target
──────  ──────────────  ──────────────────────────────────────
$7E     $0000–$FFFF     WRAM (low 64KB)
$7F     $0000–$FFFF     WRAM (high 64KB)

$00–$3F $0000–$1FFF     WRAM mirror (first 8KB)
        $2100–$213F     PPU registers (write: $2100–$2133, read: $2134–$213F)
```

```
         $2140–$217F     APU I/O ports (4 bytes, mirrored)
         $2180–$2183     WRAM data port (sequential R/W)
         $4016           Joypad serial
         $4200–$42FF     CPU internal registers (NMITIMEN, timers, math, etc.)
         $4300–$437F     DMA channel registers (8 channels × 16 bytes)
         $8000–$FFFF     Cartridge ROM (LoROM: bank × 0x8000 + addr − 0x8000)

$40–$6F  $8000–$FFFF     ROM (extended banks)

$70–$7D  $0000–$7FFF     SRAM (battery-backed save RAM)
         $8000–$FFFF     ROM
```

### MMIO Highlights

- **$4202/$4203** (WRMPYA/B): Writing the multiplier triggers immediate 8×8→16 multiplication
- **$4204–$4206** (WRDIV/B): Writing the divisor triggers immediate 16÷8 division (0 → $FFFF)
- **$420B** (MDMAEN): Writing triggers general DMA immediately, adding cycles to `pending_dma_cycles`
- **$4210** (RDNMI): Read-clears the NMI flag; bit 1 = CPU version (hardcoded 2)
- **$4211** (TIMEUP): Read-clears the IRQ flag

**`is_pure_memory(bank, addr) -> bool`**

Returns true for WRAM, SRAM, and ROM — addresses with no read side-effects. Used by the CPU idle-skip feature to determine whether a polling loop is safe to fast-forward.

---

## Module 5: DMA Engine

**File:** `src/dma.rs` + DMA methods in `src/bus.rs`

### What It Does

The SNES has 8 DMA channels, each capable of high-speed transfers between the A-bus (CPU address space) and B-bus (PPU/APU registers). Two modes: General DMA (bulk transfer) and HDMA (per-scanline register writes for raster effects).

### Structure

```
pub struct DmaChannel {
    control: u8,        // direction, indirect, decrement, fixed, transfer mode
    dest: u8,           // B-bus register offset from $2100
    src_addr: u16,      // A-bus address
    src_bank: u8,       // A-bus bank
    size: u16,          // byte count (general) / indirect addr (HDMA)

    // HDMA runtime
    hdma_indirect_bank: u8,
    hdma_addr: u16,         // table pointer
    hdma_line_counter: u8,  // bit 7 = continuous, bits 0-6 = remaining lines
    hdma_terminated: bool,
    hdma_do_transfer: bool,
}
```

### Transfer Modes

8 patterns controlling which B-bus registers are written per unit:

| Mode | Pattern | Bytes | Typical Use |
|---|---|---|---|
| 0 | [dest] | 1 | Single register |
| 1 | [dest, dest+1] | 2 | VRAM data (low+high) |
| 2 | [dest, dest] | 2 | OAM/CGRAM |
| 3 | [dest, dest, dest+1, dest+1] | 4 | — |
| 4 | [dest, dest+1, dest+2, dest+3] | 4 | — |
| 5 | [dest, dest+1, dest, dest+1] | 4 | — |

### General DMA

Triggered by writing $420B. For each enabled channel: 1. Transfer `size` bytes (0 = 65536) between A-bus and B-bus 2. Direction: A→B (CPU memory → PPU register) or B→A 3. A-bus address auto-increments/decrements unless fixed 4. Each byte costs 8 master cycles, accumulated in `pending_dma_cycles`

### HDMA

Per-scanline register animation (scroll effects, palette cycling, window position changes):

```
Frame start: hdma_init_frame()
│  For each enabled channel:
│  └── Set table pointer to src_addr, load first entry
│
Each scanline: hdma_run_scanline()
│  For each active channel:
│  ├── If hdma_do_transfer: write DMA_TRANSFER_SIZES[mode] bytes to B-bus
│  ├── Decrement line counter (bits 0-6)
│  ├── If counter == 0: load next table entry
│  └── Bit 7 = continuous → transfer every line, not just first
```

HDMA runs after the CPU finishes each scanline but before the PPU renders it. This is slightly inaccurate (real hardware interleaves HDMA mid-scanline) but correct enough for most games.

---

## Module 6: Cartridge

**File:** `src/rom.rs`

### What It Does

Parses ROM files, detects the memory mapping mode, extracts metadata, and provides read access to ROM and battery-backed SRAM.

### Structure

```
pub struct Cartridge {
    pub rom: Vec<u8>,            // raw bytes (copier header stripped)
    pub sram: Vec<u8>,           // battery-backed save RAM
    pub title: String,           // 21-byte ASCII from header
    pub map_mode: MapMode,       // LoROM or HiROM
    pub rom_size: usize,         // bytes
    pub ram_size: usize,         // bytes
    pub country: u8,
    pub checksum: u16,
    pub checksum_complement: u16,
}
```

### Header Parsing

ROM header is at offset $7FC0 (LoROM):

```
Offset  Size  Field
$00     21    Title (ASCII, space-padded)
```

```
$15     1     Map mode (bit 0: 0=LoROM, 1=HiROM)
$17     1     ROM size code (actual = 1024 << code)
$18     1     RAM size code (0 = no SRAM)
$19     1     Country
$1B     1     Version
$1C     2     Checksum complement (LE)
$1E     2     Checksum (LE)
```

Validates `checksum + complement == 0xFFFF` (warning if not). Strips 512-byte copier header if `file_size % 1024 == 512`. Attempts to load a `.srm` sidecar file for persistent SRAM.

### Read Path

LoROM formula: `offset = (bank & 0x7F) × 0x8000 + (addr − 0x8000)`. Returns 0 for out-of-range (open bus approximation).

**Note**: `MapMode::HiROM` is parsed in the enum but the bus only implements LoROM address decoding.

---

## Module 7: Frontend (Frame Loop + WASM + Browser)

**Files:** `src/lib.rs`, `src/main.rs`, `web/index.html`, `web/emulator-worker.js`, `web/index-phase-b.html`, `web/bench.html`, `web/serve.py`

### What It Does

Ties everything together: the frame loop drives CPU/PPU/APU synchronization, the WASM API exposes the emulator to JavaScript, and the browser frontend handles rendering, audio, and input.

### Frame Loop (`run_frame_inner`)

The heart of the emulator. One NTSC frame = 262 scanlines × 1364 master cycles:

```
for scanline in 0..262:
│
├── scanline == 0:
│   ├── Clear vblank, nmi_flag
│   └── hdma_init_frame()
│
├── scanline == 225 (VBLANK_START):
│   ├── Set vblank = true, nmi_flag = true
│   ├── If NMI enabled (nmitimen bit 7): cpu.nmi_pending = true
```

```
│   └── Clear auto_joypad_busy
│
├── Compute V/H-count IRQ based on nmitimen bits 5-4
│
├── CPU execution loop:
│   │  target = cpu.cycles + 1364
│   │  hblank_start = target - 272
│   │
│   │  while cpu.cycles < target:
│   │  ├── elapsed = cpu.step(&mut bus)
│   │  ├── cpu.cycles += elapsed
│   │  ├── Drain pending_dma_cycles (add to cpu.cycles + APU catch-up)
│   │  └── bus.apu.catch_up(elapsed)
│   │
│   └── Set hblank when cycles pass hblank_start
│
├── scanline 1–224 (visible):
│   ├── bus.hdma_run_scanline()
│   └── bus.ppu.render_scanline(scanline - 1)
│
└── End of frame:
    ├── frame_count += 1
    └── Convert frame_buffer (ARGB u32) → rgba_buffer (RGBA u8)
```

## WASM API

The `Emulator` struct is `#[wasm_bindgen]` and exposes:

| Method | Purpose |
|---|---|
| `new(rom_data)` | Construct from ROM bytes |
| `run_frame_no_return()` | Run one frame (zero-copy) |
| `framebuffer_ptr()` / `framebuffer_len()` | Pointer into WASM linear memory for canvas |
| `audio_samples_ptr()` / `audio_samples_len()` | Pointer for audio samples |
| `clear_audio_samples()` | Reset sample buffer after JS reads it |
| `set_button(button, pressed)` | Route input to joypad |
| `snapshot()` / `restore_snapshot()` | Save/load state |
| `set_trace(enabled)` | Toggle CPU instruction trace |

### Browser Rendering Pipeline

```
requestAnimationFrame loop:
│
├── Accumulate time (NTSC: 1000/60.0988 ms per frame, up to 2 per rAF)
│
├── emulator.run_frame_no_return()
│
├── Video (zero-copy):
│   ├── fbPtr = emulator.framebuffer_ptr()
│   ├── new Uint8ClampedArray(wasm.memory.buffer, fbPtr, fbLen)
│   ├── new ImageData(clamped, 256, 224)
│   └── ctx.putImageData(imageData, 0, 0)
│
├── Audio (zero-copy → ring buffer):
│   ├── audioPtr = emulator.audio_samples_ptr()
│   ├── new Int16Array(wasm.memory.buffer, audioPtr, audioLen)
│   ├── Convert i16 → float32 (×4 gain)
│   ├── Push to 65536-sample ring buffer (cap at 4096 to prevent drift)
│   └── emulator.clear_audio_samples()
│
└── ScriptProcessor (32 kHz, stereo):
    └── Pulls from ring buffer into AudioBuffer
```

Audio uses `createScriptProcessor(1024, 0, 2)` at 32000 Hz. AudioContext starts suspended (browser autoplay policy) and resumes on first user interaction.

### Phase B: Web Worker Architecture

`emulator-worker.js` moves emulation off the main thread: - Worker: runs `Emulator`, calls `run_frame_no_return()` at 60 Hz via `setInterval` - Transfers framebuffer + audio samples via `postMessage` with `Transferable` buffers - Main thread only paints canvas and feeds audio ring buffer - Requires `serve.py` with COOP/COEP headers for `SharedArrayBuffer` support

### Native Binary (`main.rs`)

Standalone test harness: runs 7500 frames of Zelda ALTTP with hardcoded button inputs to navigate menus. Dumps VRAM/OAM/CGRAM at frame 2550, optionally writes PPM frame dumps. Does not use the `Emulator` wrapper — drives `Cpu` + `Bus` directly. Simpler APU sync (once per scanline vs incremental).

---

## Cross-Cutting Concerns

### Save States (`snapshot.rs`)

Hand-rolled binary serialization (no serde). Format: 8-byte magic + version byte + length-prefixed blobs for CPU, Bus (containing PPU/DMA/Joypad/APU sub-blobs), and frame count. ROM is excluded (immutable). The PPU framebuffer IS included for mid-frame precision.

### Controller Input (`joypad.rs`)

16-bit button state with serial read protocol: strobe latch snapshots state, then 16 bits are read out MSB-first. Auto-joypad ($4218) exposes `current` directly without polling.

### Idle-Loop Detection (Feature: `idle-skip`)

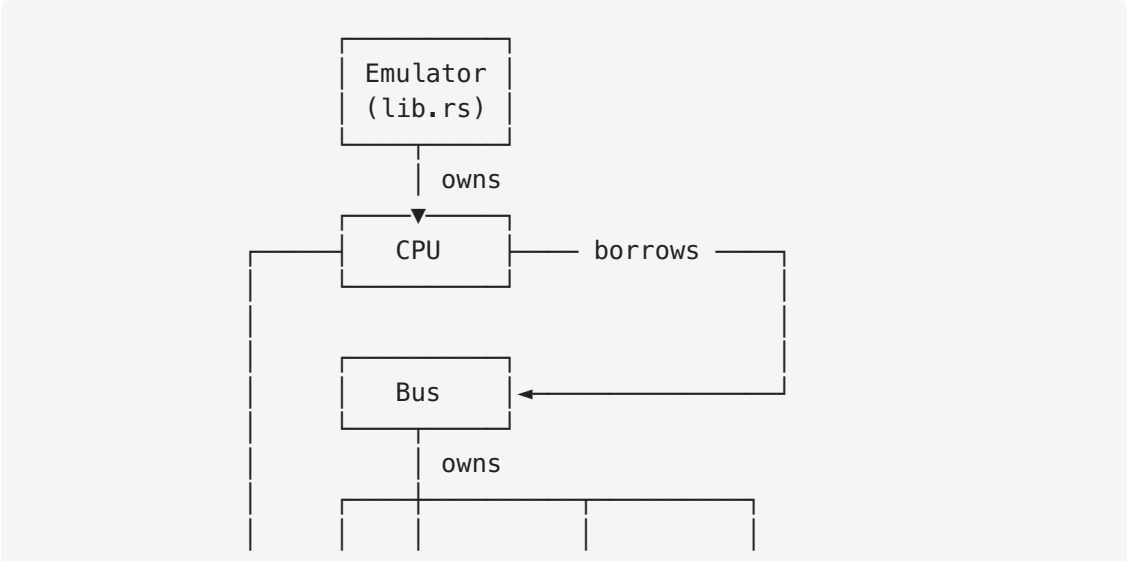
The CPU detects canonical spin-wait loops (`LDA dp; BEQ −4`) polling pure memory and fast-forwards to the scanline boundary, driving the APU in simulated chunks. Off by default due to a known audio hash divergence.

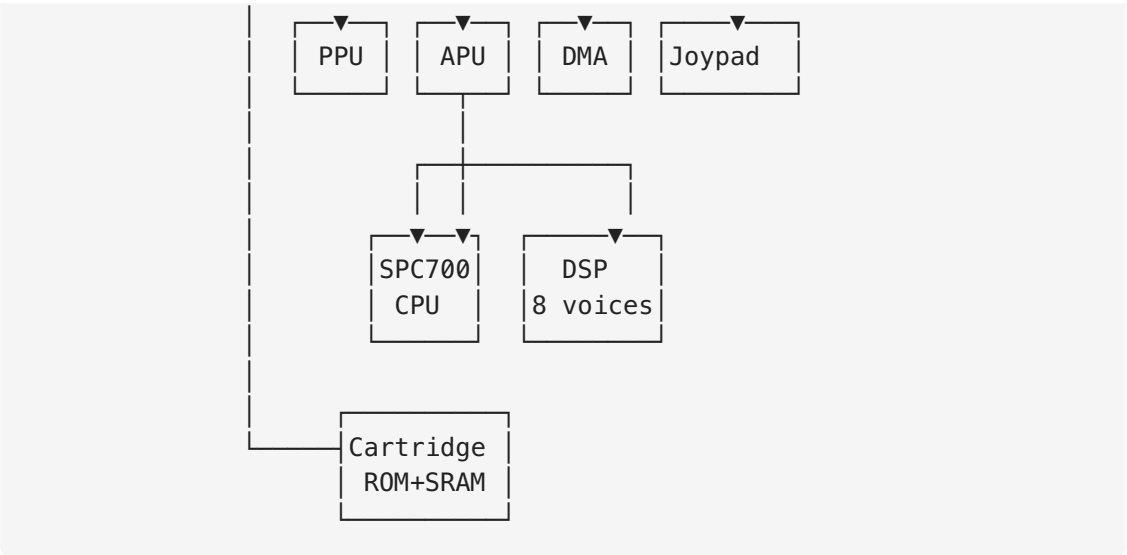
### Audio Determinism

FNV-1a 64-bit hash computed Rust-side on every audio sample drained. The browser benchmark compares this against native runs to verify WASM produces bit-identical audio.

---

## Inter-Module Dependency Graph






**Ownership**: `Emulator` owns `Cpu` + `Bus`. `Bus` owns everything else (PPU, APU, DMA, Joypad, Cartridge). `APU` owns its own `Spc700` + `ApuBus` + `Dsp` + `Timers`.

**Data flow**: CPU borrows `&mut Bus` for every instruction. Bus dispatches to sub-modules by address range. No trait abstractions, no `Rc` / `Arc`, no callbacks — pure structural ownership with mutable borrows.

**Clock synchronization**: The frame loop is the single clock source. CPU `step()` returns elapsed master cycles. APU `catch_up()` converts master→SPC cycles and runs the audio subsystem to match. PPU is rendered in bulk at scanline boundaries. DMA cycles are added to CPU's cycle count.